

Building a FuSa Documentation Automation Machine

Luke Renchik
Independent

19 April 2026

Abstract

Functional Safety evaluation and documentation are problems of data synthesis and data reasoning, which are prone to human error and require large ramp-up efforts. The goal of this tool is to synthesize, chunk, and categorically store all relevant information related to a product and leverage the use of LLM reasoning systems to assist in the evaluation and creation of the necessary documentation for these safety critical systems.



Contents

1	Part I — Foundations	2
1.1	Designing Data-Intensive Systems	2
1.1.1	Networking Protocols	2

1 Part I — Foundations

Prerequisites and background knowledge required before implementation.

1.1 Designing Data-Intensive Systems

Started: 4/19/26

1.1.1 Networking Protocols

TCP (Transmission Control Protocol).

TCP operates at the transport layer. Its job is to take raw IP packets — which are inherently unreliable and unordered — and deliver a reliable, ordered byte stream between two endpoints.

Connection Lifecycle

1. Three-Way Handshake — establishes a connection before any data flows:

```
Client → SYN      (I want to connect, my seq# is X)
Server → SYN-ACK   (OK, my seq# is Y, I acknowledge X)
Client → ACK       (Acknowledged Y)
```

2. Data Transfer

- TCP numbers every byte with a sequence number
- Receiver sends ACKs confirming what was received
- Sender retransmits anything not ACKed within a timeout
- Reorders out-of-order segments using sequence numbers before handing to the application

TCP throttles transmission rate via:

- **Receive window** — how much the receiver can buffer
- **Congestion window** — how much the network can handle

3. Teardown — uses a four-way FIN/ACK exchange. Either side can initiate.

—

HTTP (Hypertext Transfer Protocol).

HTTP is the application protocol on top of TCP. It doesn't know or care about packets — it just writes bytes to a TCP socket and reads bytes back.

Request / Response Format

```
GET /api/users HTTP/1.1
Host: example.com
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 83
```

```
{"users": [...]}
```

HTTP Versions

Speed	Slower due to overhead	Faster, minimal overhead
Use Cases	HTTP, SSH, email, file transfer	Video streaming, gaming, DNS, VoIP, QUIC

UDP is essentially raw IP with port numbers — speed and control, but you implement any reliability you need yourself (which is exactly what QUIC does).

—

QUIC.

Transport protocol built on UDP, designed by Google and standardized in RFC 9000. HTTP/3 is HTTP/2's semantics running over QUIC instead of TCP.

- **Stream isolation** — packet loss only affects the one stream it belonged to
- **Connection IDs** — instead of IP + port tuples, so connections survive switching from WiFi to LTE
- **TLS 1.3 mandatory and baked in** — handshake is combined with the transport handshake, reducing round trips
- **0-RTT resumption** — for returning connections, data can flow before the handshake even completes

—

TLS 1.3.

Encryption layer beneath HTTPS. Version 1.3 (2018) was a significant cleanup over 1.2.

- **Faster handshake** — TLS 1.2 required 2 round trips before data could flow. TLS 1.3 does it in 1 round trip. With 0-RTT resumption, a returning client can send data with the very first packet.
- **Removed legacy cruft** — TLS 1.2 supported dozens of cipher suites including broken ones (RC4, MD5, SHA-1). TLS 1.3 cuts to 5 modern suites only.
- **Forward secrecy is mandatory** — if your private key is compromised in the future, recorded past sessions still can't be decrypted, because each session used a fresh ephemeral key.